

WEEK 3 PROGRESS REPORT

Context Engineering and Retrieval-Augmented Generation (RAG)

Project: University Student-Support Case Agent

Course: BSE4104 – Emerging Trends in Software Engineering

Week: Week 3

Period: 14th September – 18th September 2026

Group: Group G

1. Introduction

During Week 3, the team focused on Context Engineering and Retrieval-Augmented Generation (RAG) for the University Student-Support Case Agent. The main objective was to move from the manually supplied context used during Week 2 toward a system that could retrieve relevant university information from an approved knowledge base before generating a response.

The team prepared a collection of approved university information, divided the documents into retrievable chunks, generated vector embeddings, implemented similarity-based retrieval, and connected the retrieved evidence to the foundation model. The system was also evaluated using a structured set of 15 student-support questions covering answerable, partially answerable, and unanswerable scenarios.

The evaluation was used not only to determine whether the system could retrieve relevant information, but also to determine whether the final answers were correctly grounded in the retrieved evidence.

2. Activities Completed

2.1 Knowledge Base Preparation

The approved university information selected for the project was prepared for use as the RAG knowledge base.

The knowledge sources covered areas including:

- Academic calendar information
- Student accommodation
- Admission information
- Counselling and guidance
- Courses at Makerere University
- Fees and payment information
- Grading information
- Graduation requirements

- Health and medical services
- Examination policies
- Student registration
- Student support directories
- Accessing university timetables

The documents were divided into smaller text sections called chunks so that relevant information could be retrieved when a student submits a question.

The resulting knowledge base contained 163 document chunks.

Each chunk retained metadata including the document title, source ID, chunk ID, source URL, and text content. This allowed retrieved information to be traced back to its original approved source.

2.2 Embedding and Retrieval Implementation

A local embedding model, `sentence-transformers/all-MiniLM-L6-v2`, was integrated into the RAG pipeline.

The model was used to convert the 163 document chunks into numerical vector representations. When a student submits a question, the question is also converted into an embedding.

The system then calculates cosine similarity between the question embedding and the document embeddings. The chunks with the highest similarity scores are ranked as the most relevant results.

The current implementation retrieves the top three chunks and passes them to the generation model as the retrieved context.

The implemented retrieval flow is therefore:

Student Question → Query Embedding → Similarity Calculation → Ranked Chunks → Top-3 Retrieved Context → Foundation Model → Final Answer

This established the basic retrieval component required for the RAG architecture.

2.3 Context Engineering and Grounding

The retrieved chunks were structured before being provided to the foundation model.

For each retrieved chunk, the context included:

- Source title
- Source ID
- Source URL
- Retrieved content

The generation instructions were also strengthened to ensure that the model used the retrieved evidence as the authoritative context for university-specific questions.

The model was instructed to:

- Use only the retrieved university information.
- Avoid using general model knowledge to fill missing information.
- Avoid inventing university policies, procedures, dates, fees, requirements, or other university-specific information.
- Clearly state when the retrieved evidence is insufficient.
- Identify the source or sources used.
- Avoid making admissions, grading, fee, disciplinary, or other high-impact university decisions.

This established a distinction between retrieval of knowledge and generation of language, while maintaining the information boundaries developed during Week 2.

2.4 Generation Model and Fallback Testing

Gemini 3.7 Flash remained the primary generation model used for the RAG implementation.

During testing, the Gemini API occasionally returned a temporary HTTP 503 high-demand error. To allow development and evaluation to continue, a fallback generation path was implemented using Groq with the **openai/gpt-oss-120b** model.

The fallback mechanism allowed the retrieved context and grounding instructions to be passed to another generation model when the primary model was temporarily unavailable.

This also demonstrated the importance of separating the retrieval component from the generation component so that a temporary generation-service failure does not prevent the RAG pipeline from being tested.

2.5 Fifteen-Case RAG Evaluation

A structured evaluation containing **15 student-support questions** was conducted.

The cases were divided into three groups:

- Q1–Q5: Answerable questions where the approved knowledge base was expected to contain sufficient information.
- Q6–Q10: Partially answerable questions where some information was expected to be available while other requested information might be missing.
- Q11–Q15: Unanswerable questions where the system was expected to avoid making unsupported claims.

For each test case, the team recorded:

- Student question
- Retrieved chunks
- Similarity scores

- Source documents
- Final generated answer
- Grounding status
- Retrieval or generation problems

The evaluation demonstrated that the system could generally distinguish between information that was available in the knowledge base and information that was not available.

For example, the system correctly avoided inventing:

- A current Bachelor of Software Engineering tuition fee when it was not present in the approved sources.
- The Academic Registrar's phone number when it was unavailable.
- The identity of a future Dean when the knowledge base did not contain the information.
- A student's personal examination timetable.
- A student's personal admission status.

2.6 RAG Evaluation Findings

The evaluation also identified several weaknesses in the current retrieval and grounding implementation.

One issue occurred in the question about the 2026/2027 Semester One dates. The retrieved Academic Calendar clearly distinguished Fresher's Orientation Week from Semester One. However, the generated response mixed the two and presented the orientation start date as an alternative Semester One start date. This demonstrated that retrieving the correct evidence does not automatically guarantee that the generated response will preserve the evidence accurately.

A second issue occurred with the first-year registration procedure. Several relevant registration chunks were retrieved, but the top-three context did not contain the complete procedure. This demonstrated that useful information can be distributed across multiple chunks and that a small top-k value can result in incomplete context.

A third issue occurred with the grading question. The exact examination-policy chunk containing the undergraduate 50% Pass mark appeared in one retrieval run but was not consistently included in another top-three result. This demonstrated instability caused by relying on a very small semantic-retrieval context.

A fourth issue occurred with the university hospital services question. Relevant hospital information was retrieved, but the detailed service information was not completely included in the final context. This indicated that information belonging to the same document section can become separated during chunking.

The evaluation therefore showed that retrieval quality and chunk structure are important factors in final-answer quality.

3. Challenges Encountered

One challenge encountered during Week 3 was that relevant information was sometimes distributed across multiple document chunks. Although several chunks could be individually relevant to a question, retrieving only the top three did not always provide enough context to answer the question completely.

This was particularly evident in the registration and hospital-service questions. The problem demonstrated the limitation of using a fixed small top-k value without considering neighbouring chunks or the structure of the original document.

Another challenge was retrieval instability. The grading question demonstrated that the same knowledge base could produce different top-three results between retrieval runs. An important chunk containing the Pass-mark evidence appeared in one run but was absent from another.

A further challenge was distinguishing between retrieval failure and generation failure. In the Semester One question, the correct evidence was successfully retrieved, but the final answer still combined two different dates. This showed that RAG evaluation must examine both the retrieved evidence and the final generated answer.

Temporary Gemini API availability issues were also encountered during testing. The existing fallback mechanism allowed evaluation to continue using the Groq `openai/gpt-oss-120b` model.

4. Deliverables Completed

The following Week 3 outputs were completed:

1. Approved university knowledge base prepared for RAG.
2. 163 processed document chunks.
3. Local embedding implementation using `sentence-transformers/all-MiniLM-L6-v2`.
4. Cosine-similarity retrieval implementation.
5. Top-k document retrieval.
6. Retrieved source metadata and traceability information.
7. Context construction for the generation model.
8. Grounding instructions for retrieved university evidence.
9. Gemini 3.7 Flash RAG integration.
10. Groq fallback generation using `openai/gpt-oss-120b`.
11. 15-case RAG evaluation.
12. Retrieval similarity scores and source records.
13. Final-answer grounding assessment.

14.Retrieval and grounding failure analysis.

15.Week 3 progress documentation.

5. Learning Outcomes

By the end of Week 3, the team gained practical experience with the main components of a Retrieval-Augmented Generation system.

The team established that RAG is not simply a matter of sending documents to a foundation model. The system requires a complete pipeline involving document preparation, chunking, embeddings, retrieval, context construction, generation, source attribution, and evaluation.

The evaluation also demonstrated the importance of context quality. A foundation model can only reliably use information that is actually supplied to it, and incomplete retrieval can therefore lead to incomplete answers.

The team also learned that successful retrieval does not automatically guarantee a correctly grounded answer. The Q1 date error demonstrated that a model can receive the correct evidence but still combine or interpret information incorrectly.

The Week 3 evaluation therefore reinforced the need for both retrieval evaluation and answer-grounding evaluation.

The work also established a distinction between:

Knowledge retrieval — finding relevant approved information.

Context engineering — selecting and structuring the information given to the model.

Generation — producing a natural-language response from the supplied context.

Grounding evaluation — checking whether the final response is actually supported by the retrieved evidence.

6. Next Steps

In Week 4, development will progress from the RAG knowledge layer toward the tool and agentic workflow components of the system.

The retrieval component will also be improved based on the Week 3 findings. Planned improvements include:

- Increasing the number of retrieved candidates beyond the current top-three approach.
- Investigating neighbouring-chunk retrieval.
- Improving section-aware document chunking.
- Investigating reranking and hybrid retrieval.
- Adding stronger grounding acceptance tests.
- Improving handling of structured information such as dates, fees, and requirements.

- Re-running the 15-case evaluation after retrieval improvements.

The team will then begin defining the bounded application tools required for the student-support case workflow, including tool permissions, inputs, outputs, and conditions under which the agent is allowed to use each tool.

7. Conclusion

Week 3 successfully established the initial Context Engineering and Retrieval-Augmented Generation layer for the University Student-Support Case Agent.

The team prepared an approved knowledge base containing 163 document chunks, implemented local embeddings and cosine-similarity retrieval, integrated retrieved context with Gemini 3.7 Flash, and implemented a Groq fallback for temporary generation-service failures.

The 15-case evaluation provided evidence that the system can retrieve and use approved university information and, importantly, can decline to provide unsupported information in many cases. At the same time, the evaluation identified weaknesses in retrieval depth, chunking, retrieval stability, and final-answer grounding.

These findings provide specific areas for improvement before the system progresses further toward the bounded agentic workflow. The Week 3 work therefore establishes the knowledge and retrieval layer that will support the tool-use and multi-step agent behaviour developed in the following stages.